RAG-Based Customer Support Assistant

Low Level Design (LLD)

Name: Devyani S. Bharare

College: P R Pote Patil College Of Engineering And Management, Amravati

Course: Internship Project

Date: April 2026

📘 1. Module-Level Design

🔹 1. Document Processing Module

- Loads PDF using PyPDFLoader
- Extracts raw text

🔹 2. Chunking Module

- Splits text into chunks
- Chunk size: 300–500 tokens
- Overlap: 50 tokens

🔹 3. Embedding Module

- Converts chunks into vector embeddings
- Uses OpenAI / HuggingFace embeddings

🔹 4. Vector Storage Module

- Stores embeddings in **ChromaDB**
- Supports similarity search

◈ 5. Retrieval Module

- Accepts query embedding
- Retrieves top-K similar chunks (k=3–5)

◈ 6. Query Processing Module

- Takes user query
- Sends to retriever + LLM
- Generates response

◈ 7. Graph Execution Module (LangGraph)

- Controls flow using nodes and edges
- Maintains state

◈ 8. HITL Module

- Handles escalation
- Allows human response

📖 2. Data Structures

📄 Document Object

```
{
  "doc_id": "string",
  "content": "text"
}
```

✂️ Chunk Object

```
{
  "chunk_id": "string",
  "text": "string",
  "embedding": [vector]
}
```

🔍 Query Object

```
{
  "query": "string",
  "embedding": [vector]
}
```

🔄 State Object (LangGraph)

```
{
  "query": "string",
  "retrieved_chunks": [],
  "answer": "string",
  "confidence": 0.0,
  "status": "processing / complete / escalate"
}
```

📘 3. Workflow Design (LangGraph)

🔹 Nodes:

- Input Node
- Processing Node
- Output Node

◈ **Flow:**

Input → Retrieve → LLM → Decision → Output / HITL

◈ **State Flow:**

- Input Node → stores query
- Processing Node → updates retrieved_chunks + answer
- Decision Node → updates confidence + status

📘 4. Conditional Routing Logic

Condition	Action
Confidence > 0.7	Send Answer
Confidence < 0.7	Escalate
No Chunks Found	Escalate
Complex query	Escalate

◈ **Confidence Calculation (simple logic)**

- Based on:
    - similarity score
    - LLM response quality

📘 5. HITL Design

◈ **When triggered:**

- Low confidence
- Missing context
- Complex queries

◈ Flow:

Query → Human Agent → Answer → Return to User

◈ Integration:

- Human response stored for future improvement

📘 6. API / Interface Design

◈ Input Format

```
{
  "query": "What is refund policy?"
}
```

◈ Output Format

```
{
  "answer": "Refund allowed within 7 days...",
  "confidence": 0.82,
  "status": "success"
}
```

◈ Escalation Response

```
{
  "status": "escalated",
  "message": "Forwarded to human agent"
}
```

📖 7. Error Handling

🔹 **Cases:**

- Empty PDF → reject upload
- No chunks → fallback message
- LLM failure → retry
- API failure → error response